

Universidade Federal de Sergipe

Biblioteca de Algoritmos

Greedy

Grupo de Estudos em Programação Competitiva

Contents

1 Binary Search and Ternary Search

1.1	BinarySearch (3325ba)	1
1.2	BinarySearchDouble (758367)	1
1.3	LowerUpperBound (4d5e64)	1

2 Dynamic Programming

2.1	Knapsack (635934)	1
2.2	LCS (6490f6)	1
2.3	LIS BS (78d63a)	2
2.4	LIS DS (12492d)	2

3 Geometry

3.1	ConvexHull (e8ad2a)	3
3.2	GeometriaInt (e08340)	4
3.3	SweepLine (9d88c7)	4

4 Graph

4.1	BFS (ecec15)	5
4.2	BinaryLifting (b9bcc)	5
4.3	Bipartite (3e995c)	5
4.4	Bridges (d768b0)	6
4.5	Cycles (b74aae)	6
4.6	DFS (bf2c67)	7
4.7	Diameter (efdec1)	7
4.8	Dijkstra (df96ee)	7
4.9	EulerTour (865bc4)	7
4.10	FindingConnectComponents (7a0e22)	8
4.11	FloydWarshall (f8ebbc)	8
4.12	LCA (35e6a7)	8
4.13	SizeOfAllSubtrees (a2922a)	9
4.14	TopoSortBFS (87f16a)	9
4.15	TopoSortDFS (b1833e)	9

5 Math

5.1	Combinacao (029489)	9
5.2	Crivo (a6487b)	9
5.3	Divisores (7c1b13)	10
5.4	Divisores2 (de6b7b)	10
5.5	Fatoracao (c23ee5)	10
5.6	LargestPrimeFactor (a499a6)	10
5.7	Modulo (8c04ba)	11

6 Strings

6.1	KMP (c41d90)	11
-----	--------------	----

7 Data Structures

7.1	DSU (ed38b0)	11
7.2	FenwickTree (27422d)	11
7.3	MinQueue (ed1384)	12
7.4	MinQueueAggStack (3748a1)	12
7.5	Mo (77aa1b)	12
7.6	ModInt (7a26a6)	13
7.7	OrderStatisticSet (1234d2)	13
7.8	SegmentTree (b19a9b)	14
7.9	SegmentTreeLazy (e2aa98)	14

8 Techniques

8.1	CoordinateCompression (eca1bb)	15
8.2	Grid (ed0740)	15

9 Teoria

9.1	STL	15
9.2	Matemática	18
9.3	Grafos	20

10 Utils

10.1	Makefile (a879a5)	21
10.2	custom hash (13b6af)	21
10.3	hash (d78ff6)	21
10.4	mistakes (06c097)	21
10.5	random (5b3cc9)	22
10.6	template (c1a851)	22
10.7	vimrc (54a8c1)	22

1 Binary Search and Ternary Search

1.1 BinarySearch (3325ba)

```
#include <bits/stdc++.h>
using namespace std;
int n, x;
bool possible(int m) { return true; }; // O(M)
// Retorna o primeiro elemento que valida nossa propriedade
// lower_bound -> primeiro valor >= x (Primeiro Verdadeiro (MINIMIZAR))
// O nosso f(x) precisa ser: [F, F, F, F, T, T, T, T], ele para quando encontrar
// o primeiro T.
int bs(int x) { // O(M*log(n))
    int ans = -1, l = 0, r = 1e18; // r = MAX
    while (l <= r) {
        int m = l + (r-l)/2; // Piso
        if (possible(m)) {
            ans = m;
            r = m - 1;
        } else {
            l = m + 1;
        }
    }
    return ans;
}
// Retorna o ultimo elemento que valida nossa propriedade
// O nosso f(x) precisa ser: [T, T, T, T, F, F, F, F], ele para quando encontrar
// o ultimo T.
int bs_last(int x) {
    int ans = -1, l = 0, r = 1e18; // r = MAX
    while (l <= r) {
        int m = l + (r-l)/2; // Teto (Ultimo Verdadeiro (MAXIMIZAR))
        if (possible(m)) {
            ans = m;
            l = m + 1;
        } else {
            r = m - 1;
        }
    }
    return ans;
}
```

1.2 BinarySearchDouble (758367)

```
#include <bits/stdc++.h>
using namespace std;
const double EPS = 1e-6; // Preciso -> defino as casas no setprecision
const int N_ITERATIONS = 100;
bool is_possible(long double m) {return true;}
long double bs(int x) {
    long double l = 0, r = 1e7;
    // for (int i = 0; i < N_ITERATIONS; i++) { -> Mais seguro
    while ((r - l) > EPS) {
        // Abaixo voce pode ajustar para o problema
        long double m = l + (r-l)/2;
        if (is_possible(m)) l = m;
        else r = m;
    }
    cout << fixed << setprecision(6) << l << endl;
}
```

1.3 LowerUpperBound (4d5e64)

```
#include <bits/stdc++.h>
using namespace std;
signed main() {
    int x; vector<int> v;
    // lower_bound: retorna um it para o PRIMEIRO elemento >= x
    auto it = lower_bound(v.begin(), v.end(), x);
    cout << "pos: " << it - v.begin() << endl;
    cout << "valor: " << *it << endl;
    // upper_bound: retorna um it para o PRIMEIRO elemento > x
    auto it = upper_bound(v.begin(), v.end(), x);
    // Quantidade de Elementos Validos = UPPER - LOWER
}
```

2 Dynamic Programming

2.1 Knapsack (635934)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1e2 + 10, MAXW = 1e5 + 10;
int n, w_max, v[MAXN], w[MAXN], dp[MAXN][MAXW];
int solve(int i, int limit) { // O(nW) -> n: num de itens, W: peso maximo
    mochila
    // Chegou no fim do array de itens
    if (i == n) return dp[i][limit] = 0; // ou INF, para min()
    // Chegou no limite de peso
    if (!limit) return dp[i][limit] = 0;
    // Ja foi calculado?
    if (dp[i][limit] != -1) return dp[i][limit];
    // Possibilidade 1: Nao adicionar o elemento i
    dp[i][limit] = solve(i+1, limit);
    // Possibilidade 2: Adicionar o elemento i
    if (limit >= w[i]) {
        // Maximo entre o valor ja calculado e a segunda possibilidade
        dp[i][limit] = max(dp[i][limit], solve(i+1, limit - w[i]) + v[i]);
    }
    return dp[i][limit];
}
signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    memset(dp, -1, sizeof(dp));
    cin >> n >> w_max;
    for (int i = 0; i < n; i++) cin >> w[i] >> v[i];
    cout << solve(0, w_max) << endl;
}
```

2.2 LCS (6490f6)

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define int long long
const int MAXN = 3e3 + 10;
string s, t;
int dp[MAXN][MAXN];
int lcs(int n, int m) { // n, m -> tamanho das strings
    if (n == 0 or m == 0) return dp[n][m] = 0;
    if (dp[n][m] != -1) return dp[n][m];
    if (s[n-1] == t[m-1]) { // Faz um match
        return dp[n][m] = 1 + lcs(n-1, m-1);
    } else { // s[n-1] != t[m-1] -> Nao fez um match
        return dp[n][m] = max(lcs(n-1, m), lcs(n, m-1));
    }
}
string get_lcs(int n, int m) {
    string str = "";
    int i = n, j = m;
```

```
while (i > 0 and j > 0) {
    if (s[i-1] == t[j-1]) {
        str += s[i-1];
        i--, j--;
    } else {
        // Caminhamos para a celula de maior valor
        if (dp[i-1][j] > dp[i][j-1]) i--;
        else j--;
    }
}
reverse(str.begin(), str.end());
return str;
}
signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    memset(dp, -1, sizeof(dp));
    cin >> s >> t;
    int n = (int) s.size(), m = (int) t.size();
    lcs(n, m);
    cout << get_lcs(n, m) << endl;
}
```

2.3 LIS BS (78d63a)

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0); cin.tie(0);
#define endl '\n'
#define all(x) (x).begin(), (x).end()

// O(n*log(n)) --> Solucao TOP
// dp[l] --> e o menor a[i] que a maior LIS de tamanho l termina
// solucao e maior l t.q d[l] nao e INF
// Obs.: Essa solucao nao serve para contar o numero de LIS da a[], precisa usar
lis-ds.

// int lis(vector<int> const& a) {
vector<int> lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> dp(n+1, INF);
    vector<int> id_dp(n+1, -1); // Armazena o indice i do valor dp[l] =
        a[i]
    vector<int> previous(n+1, -1); // Armazena o indice do dp[l-1]

    dp[0] = -INF;

    for (int i = 0; i < n; i++) {
        // Solucao trivial
        // for (int l = 1; l <= i; l++) {
        //     if (dp[l-1] < a[i] && a[i] < dp[l]) {
        //         dp[l] = a[i];
        //     }
        // }
        // }

        // dp e estritamente crescente e a[i] atualiza apenas um valor de dp[l]
        // dp[l-1] < a[i] < dp[l] --> Podemos encontrar o l a partir da Busca
            Binaria
        int l = upper_bound(all(dp), a[i]) - dp.begin();
        if (dp[l-1] < a[i] && a[i] < dp[l]) {
            dp[l] = a[i];
            id_dp[l] = i;
            previous[i] = id_dp[l-1];
        }
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (dp[l] < INF) ans = l;
    }

    // Apenas retornar a resposta
    // return ans;
```

```
// Reconstruir a subsequencia
vector<int> subseq;

int pos = id_dp[ans];
while (pos != -1) {
    subseq.push_back(a[pos]);
    pos = previous[pos];
}

reverse(all(subseq));
return subseq;
}

signed main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;

    // int ans = lis(a);
    // cout << ans << endl;

    vector<int> ans = lis(a);
    for (auto x : ans) {
        cout << x << " ";
    }
    cout << endl;
}

/* https://cp-algorithms.com/sequences/longest_increasing_subsequence.html
Considere uma array a[n].

Problema: Nosso objetivo e encontrar a maior subsequencia estritamente crescente
de a.

Subproblema: Vamos considerar que dp[l] e menor valor que um subsequencia
estritamente crescente de tamanho l termina.

- dp[0] = -INF (Caso base)

Vamos iniciar dp[l] = INF e vamos processar cada a[i] com 0 <= i <= n

Exemplo: a = {8, 3, 4, 6, 5, 2, 0, 7, 9, 1}

prefix = {} --> d = {-INF, INF, INF, ..., INF}
prefix = {8} --> d = {-INF, 8, INF, ..., INF}
prefix = {8, 3} --> d = {-INF, 3, INF, ..., INF}
prefix = {8, 3, 4} --> d = {-INF, 3, 4, ..., INF}
prefix = {8, 3, 4, 6} --> d = {-INF, 3, 4, 6, ..., INF}
prefix = {8, 3, 4, 6, 5} --> d = {-INF, 3, 4, 5, ..., INF}
prefix = {8, 3, 4, 6, 5, 2} --> d = {-INF, 2, 4, 5, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0} --> d = {-INF, 0, 4, 5, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0, 7} --> d = {-INF, 0, 4, 5, 7, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0, 7, 9} --> d = {-INF, 0, 4, 5, 7, 9, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0, 7, 9, 1} --> d = {-INF, 0, 1, 5, 7, 9, ..., INF}

Quando vamos fazer d[l] = a[i]?
- Quando tiver nenhuma lis de tamanho l que termine em a[i]
- E, se nao tiver nenhuma outra lis de tamanho l que termina em algum numero
menor que a[i]

Note que ha uma sub-estrutura otima do problema, a solucao para l pode ser
construida a partir de l - 1

dp[l] = -INF, se l = 0
min(a[i], d[l]), se a[i] > d[l - 1]
+INF, caso contrario.
*/
```

2.4 LIS DS (12492d)

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
```

```
#define all(x) (x).begin(), (x).end()

typedef long long ll;

const ll MOD = 1e9 + 7;

// Coordinate Compression
void coordinate_compression(vector<int> &a) {
    set<int> s; // Conjunto para armazenar todos os numeros unicos
    for (auto x : a) s.insert(x);

    int index = 0;
    map<int, int> mp; // Map para armazenar os novos elementos

    set<int>::iterator itr;
    for (itr = s.begin(); itr != s.end(); itr++) {
        index++;
        mp[*itr] = index;
    }

    // Alterando o valor de a
    for (int i = 0; i < a.size(); i++) {
        a[i] = mp[a[i]];
    }
}

// BIT ou SegTree
struct fenw {
    int n;
    vector<int> bit;

    fenw() {}
    fenw(int size) {
        n = size;
        bit.assign(size + 1, 0);
    }
    // query do maior prefixo a[0...r]
    int qry(int r) {
        int ans = 0;
        for (int i = r + 1; i > 0; i -= i & -i) // i & -i retorna os bits menos
            signativos de i
            ans = max(ans, bit[i]);
        return ans;
    }

    // atualiza o valor a[r] = x
    void upd(int r, int x) {
        for (int i = r + 1; i <= n; i += i & -i)
            bit[i] = max(bit[i], x);
    }
};

// O(nlog(n))
int lis(vector<int> &a) {
    coordinate_compression(a);

    int n = a.size();

    fenw tree(n + 1);

    int ans = 0;
    for (int i = 0; i < n; i++) {
        int best = tree.qry(a[i] - 1);
        tree.upd(a[i], best + 1);
        ans = max(ans, best + 1);
    }

    return ans;
}

signed main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;

    cout << lis(a) << endl;
}
```

```
// https://cp-algorithms.com/sequences/longest_increasing_subsequence.html#
// solution-in-on-log-n-with-data-structures
// Considere t[a[i]] = dp[i] # dp[i] e o maior l de LIS t.q termina com a[i]
/* Para calcular dp[], precisamos fazer:
ans = -INF;
Para i = 0 ate 0:
    dp[i] = max(t[0...a[i]] + 1) // Maior prefixo ate a[i]
    ans = max(ans, dp[i])
# O problema e buscar o maior prefixo de a[0..i] com a[k] mudando,
sendo 0 <= k < i. --> SegTree ou BIT
# Pontos Negativos da Solucao
- Implementacao Complexa
- a[i] for mt grande --> Coordinate Compression ou Dynamic SegTree
*/
```

3 Geometry

3.1 ConvexHull (e8ad2a)

```
#include <bits/stdc++.h>
using namespace std;

#define int long long int

struct pt {
    int x, y, id;
    pt(int x, int y, int id) : x(x), y(y), id(id) {}
    pt() {}
    pt operator-(pt const &o) const {
        return {x - o.x, y - o.y, -1};
    }
    bool operator<(pt const &o) const {
        if (x == o.x) {
            return y < o.y;
        }
        return x < o.x;
    }
    int operator^(pt const &o) const {
        return (int)x * o.y - (int)y * o.x;
    }
};

int ccw(pt const &a, pt const &b, pt const &x) {
    auto p = (b - a) ^ (x - a);
    return (p > 0) - (p < 0);
}

vector<pt> convex_hull(vector<pt> P, bool include_collinear) {
    // include_collinear: define se vai retornar os pontos colineares as
    // bordas
    // ou seja, pontos da aresta do poligono
    sort(P.begin(), P.end());
    vector<pt> L, U;
    int pop_threshold = include_collinear ? -1 : 0;
    for (auto p : P) {
        while (L.size() >= 2 && ccw(L.end()[-2], L.end()[-1], p) <=
            pop_threshold) {
            L.pop_back();
        }
        L.push_back(p);
    }
    reverse(P.begin(), P.end());
    for (auto p : P) {
        while (U.size() >= 2 && ccw(U.end()[-2], U.end()[-1], p) <=
            pop_threshold) {
            U.pop_back();
        }
        U.push_back(p);
    }
    L.insert(L.end(), U.begin() + 1, U.end() - 1);
    return L;
}

signed main() {
    int n; cin >> n;
    vector<pt> arr;
```

```
for (int i = 0; i < n; i++) {
    int x, y; cin >> x >> y;
    arr.emplace_back(x, y, i+1);
}
vector<pt> ans = convex_hull(arr, true);
for (auto [x, y, id] : ans) {
    cout << x << " " << y << " " << id << '\n';
}
}
```

3.2 GeometriaInt (e08340)

```
#include <bits/stdc++.h>
using namespace std;
/* PRIMITIVAS */
#define int long long
#define sq(x) ((x)*(int)(x))
struct pt {
    int x, y;
    pt(int a=0, int b=0) : x(a), y(b) {}
    friend istream &operator>>(istream &in, pt &p) {
        int x, y; in >> p.x >> p.y; return in;
    }
    bool operator < (const pt p) const {
        if (x != p.x) return x < p.x; return y < p.y;
    }
    bool operator == (const pt p) const {
        return x == p.x and y == p.y;
    }
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
    // multiplicar um escalar ao vetor
    pt operator * (const int c) const { return pt(x*c, y*c); }
    // produto escalar
    int operator * (const pt p) const { return x*(int)p.x + y*(int)p.y; }
    // produto vetorial (norma de u x v)
    int operator ^ (const pt p) const { return x*(int)p.y - y*(int)p.x; }
};

struct line {
    pt p, q;
    line() {}
    line(pt _p, pt _q) : p(_p), q(_q) {}
    friend istream &operator>>(istream &in, line &r) {
        int x, y; in >> r.p >> r.q; return in;
    }
};

/* PONTO e VETOR */
int dist2(pt p, pt q) { return sq(p.x - q.x) + sq(p.y - q.y); }
// area do paralelogramo formado pelos vetores p->q e p->r
// dividir por 2 retorna a area do triangulo formado pelos pontos
int area2(pt p, pt q, pt r) { return (q - p) ^ (r - p); }
// verifica se p, q, r sao colineares
bool col(pt p, pt q, pt r) { return area2(p, q, r) == 0; }
// verifica se p -> q -> r e no sentido anti-horario
bool ccw(pt p, pt q, pt r) { return area2(p, q, r) > 0; }
// quadrante de um ponto -> 1, 2, 3, 4
int quad(pt p) { return (p.x < 0) ^ 3 * (p.y < 0); }
// retorna se ang(p) < ang(q) -> Radial Sort (ordena pelos angulos)
bool compare_angle(pt p, pt q) {
    if (quad(p) != quad(q)) return quad(p) < quad(q);
    return ccw(q, pt(0, 0), p);
}

// rotaciona 90 graus o vetor
// vetor resultante e o normal ao original
pt rotate90(pt p) { return pt(-p.y, p.x); }

/* RETA */
// verifica se P pertence ao segmento R
bool in_seg(pt p, line r) {
    pt a = r.p - p, b = r.q - p;
    // (a * b) <= garante que p esta entre [r.p, r.q]
    return (a ^ b) == 0 and (a * b) <= 0;
}

// se o seg de r intersecta o seg de s
```

```

bool inter_seg(line r, line s) {
    if (in_seg(r.p, s) or in_seg(r.q, s)
        or in_seg(s.p, r) or in_seg(s.q, r)) return 1;
    return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and
           ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
}
// numero de pontos inteiros no segmento
int segpoints(line r) {
    return 1 + __gcd(abs(r.p.x - r.q.x), abs(r.p.y - r.q.y));
}
// retorna t tal que t*v pertence a reta r
double get_t(pt v, line r) {
    return (r.p.r.q) / (double) ((r.p-r.q)^v);
}

/* POLIGONOS */
// Shoelace Theorem: 2*area
int polarea2(vector<pt> v) {
    int ret = 0;
    for (int i = 0; i < v.size(); i++)
        ret += area2(pt(0, 0), v[i], v[(i+1) % v.size()]);
    return abs(ret);
}
// Verifica se um ponto esta dentro, fora ou na borda de um poligono
// 0 -> fora, 1 -> dentro e 2 -> na borda
int inpol(vector<pt>& v, pt p) { // O(|v|) -> raycasting
    int qt = 0;
    for (int i = 0; i < v.size(); i++) {
        if (p == v[i]) return 2;
        int j = (i + 1) % v.size();
        if (p.y == v[i].y and p.y == v[j].y) {
            if ((v[i]-p)*(v[j]-p) <= 0) return 2;
        }
        bool baixo = v[i].y < p.y;
        if (baixo == (v[j].y < p.y)) continue;
        auto t = (p-v[i])^(v[j]-v[i]);
        if (!t) return 2;
        if (baixo == (t > 0)) qt += (baixo ? 1 : -1);
    }
    return qt != 0;
}

```

3.3 SweepLine (9d88c7)

```

#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ENTRADA 0
#define SAIDA 1

typedef struct Evento {
    int tempo, tipo, id;
    Evento(int _tempo, int _tipo, int _id) : tempo(_tempo), tipo(_tipo), id(_id) {}
    // O elemento id e util quando temos que responder queries
    bool operator < (Evento outro) {
        if (tempo == outro.tempo) return tipo < outro.tempo; // Definir o
        // criterio de desempate
        return tempo < outro.tempo;
    }
} evento_t;

int main() {
    int n; cin >> n;
    vector<evento_t> ev;
    // Leitura dos eventos
    for (int i = 0; i < n; i++) {
        int e, s; cin >> e >> s;
        ev.push_back({e, ENTRADA, i}); // Evento de Entrada
        ev.push_back({s, SAIDA, i}); // Evento de Saida
    }
    // Pre-ordenacao para Varredura
    sort(ev.begin(), ev.end());
    // Varredura --> Verificar qual o maximo de pessoas

```

```

    int acc = 0, ans = -1;
    for (auto [tempo, tipo, id] : ev) {
        if (tipo == ENTRADA) acc++;
        else acc--;
        ans = max(ans, acc);
    }
    cout << ans << endl;
    return 0;
}

```

/*
Problemas de sweep line sao descritas por duas variaveis:
- Localizacao temporal ou espacial;
- Tipo do evento

O problema esta em saber qual a melhor estrutura para armazenar a resposta (um inteiro, um set, uma Segment Tree, ...)

<https://noic.com.br/materiais-informatica/curso/line-sweep/>
*/

4 Graph

4.1 BFS (ecec15)

```

#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 1e5 + 10;
int n, m;
vector<int> adj[MAXN], dist(MAXN, -1), parent(MAXN, -1);
bool vis[MAXN];
// BFS Multisource -> bfs(vector<pi> ms): permite fazer BFS em varias fontes
// Encontrar o menor ciclo de um grafo ou menor ciclo que possui um vertice
// Encontrar a menor distancia de s para todos os outros vertices
void bfs(int s) { // O(V + E)
    queue<int> q;
    q.push(s); vis[s] = true;
    dist[s] = 0;
    parent[s] = -1;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto u : adj[v]) if (!vis[u]) {
            q.push(u); vis[u] = true;
            dist[u] = dist[v] + 1;
            parent[u] = v;
        }
    }
}
// https://cp-algorithms.com/graph/breadth-first-search.html
signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    bfs(0);
    // Mostrar o caminho de s ate u
    int u; cin >> u;
    if (!vis[u]) {
        cout << "No path" << endl;
    } else {
        stack<int> path;
        for (int v = u; v != -1; v = parent[v]) {
            path.push(v);
        }
        // Exibir caminho
        cout << "Path: ";
        while (!path.empty()) {
            cout << path.top() + 1 << " ";
        }
        cout << endl;
    }
}

```

4.2 BinaryLifting (b9bccc)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5 + 10;
const int LOG = 20; // 2^20 ~ 1e6
int n, q, parent[MAXN], up[MAXN][LOG];
vector<int> adj[MAXN];
void binary_lifting() { // O(nlog(n))
    // Calcula o primeiro ancestral
    for (int v = 0; v < n; v++)
        up[v][0] = parent[v];
    // Calcular o 2 ate k-th ancestral
    for (int j = 1; j < LOG; j++)
        for (int v = 0; v < n; v++)
            if (up[v][j-1] == -1)
                up[v][j] = -1;
            else
                up[v][j] = up[ up[v][j-1] ][j-1];
}
int kth_ancestor(int v, int k) { // O(log(n))
    for (int j = 0; j < LOG; j++) {
        if (v == -1) break;
        if (k & (1LL << j)) v = up[v][j];
    }
    return v;
}
void solve() {
    memset(parent, -1, sizeof(parent));
    cin >> n >> q;
    for (int v = 1; v < n; v++) {
        int u; cin >> u; u--;
        adj[v].pb(u);
        adj[u].pb(v);
        parent[v] = u;
    }
    binary_lifting();
    // Responder as queries
    for (int i = 0; i < q; i++) {
        int x, k; cin >> x >> k; x--;
        int ans = kth_ancestor(x, k);
        if (ans == -1) cout << ans;
        else cout << ans+1;
        cout << endl;
    }
}
```

4.3 Bipartide (3e995c)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 2e5+5;

bool vis[MAXN];
int n, m, color[MAXN];
vector<int> adj[MAXN];

bool dfs(int v) {
    for (auto u : adj[v]) {
        if (color[u] == -1) {
            color[u] = (color[v] == 1) ? 2 : 1;
            if (!dfs(u)) return false;
        } else if (color[u] == color[v]) {
            return false; // conflito -> nao bipartido
        }
    }
    return true;
}
```

```

}

signed main() {
    memset(color, -1, sizeof(color));
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    bool valid = true;
    for (int i = 0; i < n; i++) {
        if (color[i] == -1) {
            color[i] = 1;
            if (!dfs(i)) {
                valid = false;
                break;
            }
        }
    }

    if (valid) {
        for (int v = 0; v < n; v++) cout << color[v] << " ";
        cout << endl;
    } else {
        cout << "IMPOSSIBLE" << endl;
    }
}
```

4.4 Bridges (d768b0)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 2e5 + 5;
vector<int> adj[MAXN];
vector<pair<int, int>> ans;
// dp[u] -> qtd de back-edges que passam por cima da aresta (parent[u], u)
// lvl[u] -> nivel do vertice u
int n, dp[MAXN], lvl[MAXN], parent[MAXN];
void dfs(int v) {
    dp[v] = 0;
    for (auto u : adj[v]) {
        if (u == parent[v]) continue;
        if (lvl[u] == 0) {
            parent[u] = v;
            lvl[u] = lvl[v]+1;
            dfs(u);
            dp[v] += dp[u];
        } else if (lvl[u] < lvl[v]) { // aresta acima
            dp[v]++;
        } else if (lvl[u] > lvl[v]) { // aresta abaixo
            dp[v]--;
        }
    }

    if (parent[v] != -1 and dp[v] == 0) {
        // Encontrou uma ponte
        ans.pb({min(v, parent[v]), max(v, parent[v])});
    }
}

void find_bridges() {
    memset(parent, -1, sizeof(parent));
    for (int v = 0; v < n; v++) {
        if (lvl[v] == 0) {
            lvl[v] = 1; // marca como visitado
            dfs(v);
        }
    }
}

// https://onlinejudge.org/external/7/796.pdf
/*
Ponte e uma aresta que caso seja removida transforma o grafo em desconexo
- Back-edge e uma aresta (u, v) que conecta u (pai) com v (descendente de u).
- Span-edge e uma aresta (u, v) marcada na DFS, representa uma DFS Tree do Grafo G.
*/
```

Observacoes

1) Uma span-edge (u, v) e uma ponte <-> nao existe nenhuma back-edge que conecta algum ancestral de uv

com algum descendente de uv.
<-> nao existe um back-edge que "atravessa por alto" uv.

2) A back-edge nao e nunca ponte.

Algoritmo:

Dado um grafo G:

1) Construa a DFS Tree de G

2) Para toda span-edge (u, v), caso nao existe nenhuma back-edge que "atravessa por alto" (u, v),

(u, v) e uma ponte.

*/

4.5 Cycles (b74aae)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAX = 1e5 + 10;
int n, m;
vector<int> adj[MAX];
int parent[MAX], cycle_start, cycle_end;
bool vis[MAX]; // Grafo nao-direcionado
int color[MAX]; // Grafo direcionado
// Grafo nao-direcionado
bool dfs(int v) {
    vis[v] = true;
    for (auto u : adj[v]) {
        if (u == parent[v]) continue;
        if (vis[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u)) return true;
    }
    return false;
}
// Grafo direcionado
bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

signed main() {
    cycle_start = -1;
    memset(parent, -1, sizeof(parent));
    memset(color, 0, sizeof(color));
    // Grafo nao-direcionado
    for (int v = 0; v < n; v++) {
        // break -> encontrou um ciclo
        if (!vis[v] and dfs(v)) break;
    }
    // Grafo direcionado
    for (int v = 0; v < n; v++) {
        // break -> encontrou um ciclo
        if ((color[v]==0) and dfs(v)) break;
    }
    // Exibindo o ciclo
    if (cycle_start == -1) {
```

```
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.pb(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.pb(v);
        cycle.pb(cycle_start);
    }
    // https://cp-algorithms.com/graph/finding-cycle.html
```

4.6 DFS (bf2c67)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define MAXN 100005
vector<int> adj[MAXN];
bool visited[MAXN];
int n, m;
// O(v + E)
void dfs(int v) { // dfs(int v, int p=-1) -> Arvores
    visited[v] = true;
    // for (auto u : adj[v]) if (u != p) { -> Arvores
    for (auto u : adj[v]) if (!visited[u]) {
        dfs(u);
    }
}
// https://cp-algorithms.com/graph/depth-first-search.html
```

4.7 Diameter (efdec1)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 2e5+5;
int n, dist[MAXN];
vector<int> adj[MAXN];
void dfs(int v, int p) {
    for (auto u : adj[v]) if (u != p) {
        dist[u] = dist[v] + 1;
        dfs(u, v);
    }
}

void solve() {
    cin >> n;
    for (int i = 0; i < n-1; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    dfs(0, -1);
    int d = 0, a = 0, b = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            a = i;
        }
    }
    memset(dist, 0, sizeof(dist));
    dfs(a, -1);
    d = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            b = i;
        }
    }
    cout << a << " " << b << endl;
    cout << d << endl;
```


4.8 Dijkstra (df96ee)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
typedef pair<int, int> pi;
const int INF = 0x3f3f3f3f3f3f3f11;
const int MAX = 1e5 + 10;
int n, m;
vector<pi> adj[MAX]; // {vizinho, peso}
bool vis[MAX]; int dist[MAX], parent[MAX];
void dijkstra(int s) {
    fill(vis, vis+MAX, false);
    fill(dist, dist+MAX, INF);
    // priority_queue de {distancia, vertice}
    priority_queue<pi, vector<pi>, greater<pi>> pq;
    pq.push({0, s}); dist[s] = 0; parent[s] = -1;
    while (!pq.empty()) {
        auto [d, v] = pq.top(); pq.pop();
        if (vis[v]) continue;
        vis[v] = true;
        for (auto [u, w] : adj[v]) if (dist[u] > dist[v] + w) {
            dist[u] = dist[v] + w;
            parent[u] = v;
            pq.push({dist[u], u});
        }
    }
}
signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c; a--, b--;
        adj[a].push_back({b, c});
    }
}
// https://cp-algorithms.com/graph/dijkstra.html
```

4.9 EulerTour (865bc4)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5 + 10;
int n, timer=1, tin[MAXN], tout[MAXN], euler[2*MAXN+1];
vector<vector<int>> adj;
// https://courses.edx.org/asset-v1:ITMOx+I2CPx+3T2016+type@asset+block/lecture
// -04.pdf
void dfs(int v) {
    tin[v] = timer;
    euler[timer++] = v;
    // for (auto u : adj[v]) if (u != p) { // Arvore
    for (auto u : adj[v]) if (tin[u] == -1) {
        dfs(u);
    }
    tout[v] = timer;
    euler[timer++] = v;
}
// Verifica se A e ancestral de B
bool is_ancestor(int a, int b) {
    // A e ancestral de B <-> tin[a] < tout[b] < tout[a]
    return (tin[a] < tout[b]) and (tout[b] < tout[a]);
}
int main() {
    memset(tin, -1, sizeof(tin));
    n = 9;
    adj = {
        {2, 1}, // A
        {0, 5, 6}, // B
        {0, 3, 4}, // C
        {2, 4}, // D
        {2, 3}, // E
        {1, 6}, // F
        {1, 5, 7, 8}, // G
        {6, 8}, // H
    }
```

```
        {6, 7} // J
    };
    dfs(0);
    for (int i = 0; i < n; i++) {
        char c = i+'A';
        cout << c << ": tin = " << tin[i]
            << ", tout = " << tout[i] << endl;
    }
    for (int i = 1; i < timer; i++) {
        char c = euler[i]+'A';
        cout << c << " ";
    }
    cout << endl;
    return 0;
}
```

4.10 FindingConnectComponents (7a0e22)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 1e5 + 10;
vector<int> adj[MAXN], comp;
bool vis[MAXN];
int n, m;

void dfs(int v) {
    vis[v] = true;
    comp.pb(v);
    for (auto u : adj[v]) if (!vis[u]) {
        dfs(u);
    }
}

signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    // Encontrar os componentes conexos (O(V + E))
    for (int v = 0; v < n; v++) {
        if (!vis[v]) {
            comp.clear();
            dfs(v);
            // Percorrendo no componente conexo
            cout << "Component: ";
            for (auto u : comp) {
                cout << u << " ";
            }
            cout << endl;
        }
    }
}
// https://cp-algorithms.com/graph/search-for-connected-components.html
```

4.11 FloydWarshall (f8ebbc)

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 0x3f3f3f3f;

#define MAXN 10000

// n -> numero de vertices, m -> numero de arestas
int n, m;
// w[i][j] -> matriz com o peso da aresta (i, j)
int w[MAXN][MAXN];
// dist[i][j] -> matriz com a menor distancia entre os vertices i e j
int dist[MAXN][MAXN];
```

```

void initialize() {
    for (int i = 0 ; i < n ; i++) {
        for (int j = 0 ; j < n ; j++) {
            if (i == j) {
                dist[i][j] = 0;
            } else {
                dist[i][j] = INF ;
            }
        }
    }
}

void floyd_warshall() {
    for (int k = 0 ; k < n ; k++) {
        for (int i = 0 ; i < n ; i++) {
            for (int j = 0 ; j < n ; j++) {
                dist[i][j] = min(dist[i][j] , dist[i][k] + dist[k][j]) ;
            }
        }
    }
}

signed main() {
    cin >> n >> m;
    initialize();
    for (int i = 0; i < m; i++){
        // a, b -> vertices e c -> custo da aresta (a, b)
        int a, b, c; cin >> a >> b >> c;
        w[a][b] = c;
        dist[a][b] = min(dist[a][b], c);
        // Comente as linhas abaixo, caso o Grafo seja direcionado
        w[b][a] = c;
        dist[b][a] = min(dist[b][a], c);
    }
    floyd_warshall();
}

```

4.12 LCA (35e6a7)

```

#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5+5;
const int LOG = 20;
int n, q, up[MAXN][LOG], depth[MAXN];
vector<int> adj[MAXN];
void dfs(int v, int p) { // O(nlog(n))
    for (auto u : adj[v]) if (u != p) {
        depth[u] = depth[v] + 1;
        up[u][0] = v;
        for (int j = 1; j < LOG; j++) {
            up[u][j] = up[ up[u][j-1] ][j-1];
        }
        dfs(u, v);
    }
}

// Binary Lifting -> Encontrar o kth ancestor O(logn)
int get_kth(int x, int k) {
    if (k > depth[x]) return -1;
    for (int i = 0; i < LOG; i++) {
        if (k & (1 << i))
            x = up[x][i];
    }
    return x;
}

// Encontrar o LCA O(log(n))
int get_lca(int a, int b) {
    if (depth[a] < depth[b]) {
        swap(a, b);
    }
    int k = depth[a] - depth[b];
    for (int j = LOG-1; j >= 0; j--) {
        if (k & (1 << j)) {
            a = up[a][j];
        }
    }
    if (a == b) {

```

```

        return a;
    }
    for (int j = LOG-1; j >= 0; j--) {
        if (up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}

void solve() {
    cin >> n >> q;
    for (int v = 1; v < n; v++) {
        int u; cin >> u; u--;
        adj[v].pb(u);
        adj[u].pb(v);
    }
    dfs(0, -1);
    for (int i = 0; i < q; i++) {
        // int a, b; cin >> a >> b; a--, b--;
        // cout << (get_lca(a, b) + 1) << endl;
        int x, k; cin >> x >> k; x--;
        int ans = get_kth(x, k);
        if (ans == -1) cout << ans;
        else cout << ans+1;
        cout << endl;
    }
}

```

4.13 SizeOfAllSubtrees (a2922a)

```

#include <bits/stdc++.h>
using namespace std;
#define pb push_back

const int MAXN = 1e5 + 10;

int n, m, subtree_size[MAXN];
vector<int> adj[MAXN];
bool vis[MAXN];

void dfs(int v) {
    vis[v] = true;
    subtree_size[v] = 1;
    for (auto u : adj[v]) if (!vis[u]) {
        dfs(u);
        subtree_size[v] += subtree_size[u];
    }
}

signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    dfs(0);
}

```

4.14 TopoSortBFS (87f16a)

```

#include <bits/stdc++.h>
using namespace std;
// Kahn Algorithm
// BFS + indegree: grau de incidencia
const int MAXN = 1e5 + 10;
int n, m;
vector<int> adj[MAXN], indegree(MAXN, 0), order;
void topoSort() {
    queue<int> q;
    // priority_queue<int> pq; // Ha uma preferencia nos vertices

```

```

for (int v = 0; v < n; v++) {
    if (indegree[v] == 0) q.push(v);
}
while (!q.empty()) {
    int v = q.front(); q.pop();
    // int v = pq.top(); pq.pop();
    order.push_back(v);
    for (auto u : adj[v]) {
        indegree[u]--;
        if (indegree[u] == 0) {
            q.push(u);
        }
    }
}
}
signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[u].push_back(v);
        indegree[v]++;
    }
}

```

4.15 TopoSortDFS (b1833e)

```

#include <bits/stdc++.h>
using namespace std;
// Grafo Aciclico Dirigido (DAG)
// - Arestas = Dependencias -> Ex.: Disciplinas e Pre-requisitos
// - Definicao: E uma permutacao dos vertices tal que
// para toda aresta v -> u, v aparece antes de u e "u depende de v"
// - Ordem de execucao da DP
const int MAXN=1e5+10;
int n, color[MAXN];
vector<int> adj[MAXN]; deque<int> order;
void dfs(int v) {
    color[v] = 1;
    for (auto u : adj[v]) {
        if (color[u] == 0) dfs(u);
        if (color[u] == 1) { // Achou um ciclo
            cout << "IMPOSSIBLE!" << endl;
            exit(0); // Nao e um DAG
        }
    }
    order.push_front(v);
    color[v] = 2;
}
void topoSort() {
    for (int v = 0; v < n; v++) {
        if (color[v] == 0) dfs(v);
    }
}

```

5 Math

5.1 Combinacao (029489)

```

#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100;
int comb[MAXN+1][MAXN+1];
// pre calculando as combinacoes -> O(n)
void pre() {
    comb[0][0] = 1;
    for (int n = 1; n <= MAXN; n++) {
        comb[n][0] = comb[n][n] = 1;
        for (int k = 1; k < n; k++) {
            comb[n][k] = comb[n-1][k-1] + comb[n-1][k];
        }
    }
}

```

```

    }
}
signed main() {
    pre();
}

```

5.2 Crivo (a6487b)

```

#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5+10;
bool primo[MAXN];
// Crivo ate raiz de N --> Para descobrir todos os primos ate N, basta achar
// todos os primos que nao ultrapassam N
void crivo() { // O(nlog(log(n)))
    for (int i = 0; i < MAXN; i++) primo[i] = true;
    primo[0] = primo[1] = false;
    for (int i = 2; i * i < MAXN; i++) {
        if (primo[i] and (long long) i * i < MAXN) {
            for (int j = i * i; j < MAXN; j += i) {
                primo[j] = false;
            }
        }
    }
}
// Crivo ate N
// Util quando queremos fatorar numeros ate MAXN^2
void crivo2() {
    for (int i = 0; i < MAXN; i++) primo[i] = true;
    primo[0] = primo[1] = false;
    for (int i = 2; i < MAXN; i++) {
        if (primo[i]) {
            // i e primo
            if ((long long) i * i < MAXN) {
                for (int j = i*i; j < MAXN; j+=i) {
                    primo[j] = false;
                }
            }
        }
    }
}
// https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html#implementation

```

5.3 Divisores (7c1b13)

```

#include <bits/stdc++.h>
using namespace std;
vector<int> divisores[100005];
signed main() {
    // Enumerar todos os divisores dos numeros de i ate N
    int n; cin >> n;
    for (int i = 1; i <= n; i++) { // O(nlog(n))
        for (int j = i; j <= n; j += i) { // j -> multiplo de i
            divisores[j].push_back(i);
        }
    }
}
/*
# Teorema Fundamental da Aritmetica

```

Todo inteiro $N > 1$ pode ser representado de forma unica (exceto pela ordem) como produto de numeros primos.

$$N = (p_1^{e_1}) * (p_2^{e_2}) * \dots * (p_k^{e_k})$$

Quantidade de Divisores

A quantidade de divisores de um numero N , denotada por $d(N)$, e calculada a partir de seus expoentes primos

```
d(N) = (e1 + 1)(e2 + 1)...(ek + 1)
```

Obs.: a qtd de divisores de N so e impar, se N for um QUADRADO PERFEITO!
*/

5.4 Divisores2 (de6b7b)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
vector<int> divisores(int n) { // O(sqrt(N))
    vector<int> ans;
    for (int a = 1; a*a <= n; a++) {
        if (n % a == 0) {
            int b = n / a;
            ans.push_back(a);
            if (a != b) ans.push_back(b);
        }
    }
    return ans;
}
```

5.5 Fatoracao (c23ee5)

```
#include <bits/stdc++.h>
using namespace std;
map<int, int> fatorar(int n) { // O(sqrt(n))
    map<int, int> exp;
    for (int i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            exp[i]++;
            n /= i;
        }
    }
    if (n > 1) exp[n]++;
    return exp;
}
```

5.6 LargestPrimeFactor (a499a6)

```
#include <bits/stdc++.h>
using namespace std;
// E possivel encontrar o maior fator primo de um numero
// a partir da conjectura de 'Twin Primes'
int main() {
    int n; cin >> n;
    int ans = -1;
    while (n % 2 == 0) {
        n /= 2;
        ans = 2;
    }
    while (n % 3 == 0) {
        n /= 3;
        ans = 3;
    }
    for (int i = 5; i*i <= n; i++) {
        while (n % i == 0) {
            n /= i;
            ans = i;
        }
        while (n % (i + 2) == 0) {
            n /= (i + 2);
            ans = (i + 2);
        }
    }
    if (n > 4) ans = n;
    cout << ans << endl;
}
```

5.7 Modulo (8c04ba)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
// Assuma que os inputs estao no range [0, MOD)
const int MOD = 1e9 + 7;
inline int add(int a, int b) {return ((a % MOD) + (b % MOD)) % MOD;}
inline int sub(int a, int b) {return ((a % MOD) - (b % MOD) + MOD) % MOD;}
inline int mult(int a, int b) {return ((a % MOD) * (b % MOD)) % MOD;}
inline int pow(int x, int y, int p) { // (x ^ y) % p
    int res = 1;
    x %= p;
    if (x == 0) return 0;
    while (y > 0) {
        if (y & 1)
            res = (res * x) % p;
        y = y >> 1;
        x = (x * x) % p;
    }
    return res;
}
// Calcular inverso modular: a^-1
inline int inv(int a) {return pow(a, MOD - 2, MOD);}
inline int divi(int a, int b) {return mult(a, inv(b));}
signed main() {
    int a = 10, b = 20; // Substitua o MOD para 7
    cout << add(a, b) << endl; // 2
    cout << sub(a, b) << endl; // 4
    cout << mult(a, b) << endl; // 4
    cout << divi(a, b) << endl; // 4
}
```

6 Strings

6.1 KMP (c41d90)

```
#include <bits/stdc++.h>
using namespace std;

// n = |padrao| e m = |texto|
// pi: O(n)    match: O(n + m)    automato: O(|sigma| * n)

// Funcao de prefixo pi(i):
// Para todo prefixo s' de s, a funcao calcula o tamanho do maior
// do prefixo proprio de s' que tambem e sufixo.
vector<int> pi(string s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < s.size(); i++) {
        while (j > 0 and s[i] != s[j]) j = p[j-1];
        if (s[i] == s[j]) j++;
        p[i] = j;
    }
    return p;
}

// t -> texto, s -> padrao
// A funcao retorna a quantidade de matchings
vector<int> matching(string &t, string& s) {
    // '$' e um caracter impossivel no padrao,
    // serve para nao tratar o caso de quando acaba o matching
    vector<int> p = pi(s+'$'), match;
    for (int i = 0, j = 0; i < t.size(); i++) {
        while (j > 0 and t[i] != s[j]) j = p[j-1];
        if (t[i] == s[j]) j++;
        if (j == s.size()) match.push_back(i-j+1);
    }
    return match;
}
```

7 Data Structures

7.1 DSU (ed38b0)

```
#include <bits/stdc++.h>
using namespace std;
struct dsu {
    // pai = representante do conjunto
    vector<int> parent, size;
    dsu(int n) {
        parent.resize(n);
        size.resize(n);
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }
    // find e uni: O(a(n)) ~= O(1) arrotizado
    int find(int i) {
        // Path Compression
        return parent[i] = (parent[i] == i) ? i : find(parent[i]);
    }
    void uni(int a, int b) { // o pai de a se torna pai de b
        a = find(a), b = find(b);
        if (a != b) {
            // a -> maior arvore
            if (size[a] < size[b]) { // Small to Large Otimization
                swap(a, b);
            }
            parent[b] = a;
            size[a] += size[b];
        }
    }
};
// https://cp-algorithms.com/data_structures/disjoint_set_union.html
```

7.2 FenwickTree (27422d)

```
// Binary Indexed Tree ou Fenwick Tree
#include <bits/stdc++.h>
using namespace std;

// 0-INDEXED
struct fenw {
    int n;
    vector<int> bit;
    fenw() {}
    fenw(int size) {
        n = size;
        bit.assign(size + 1, 0);
    }
    // query do prefixo a[0] + a[1] + ... + a[r]
    int qry(int r) {
        int ans = 0;
        for (int i = r + 1; i > 0; i -= i & -i) // i & -i retorna os bits menos
            signativos de i
            ans += bit[i];
        return ans;
    }
    // atualiza o valor a[r] = x
    void upd(int r, int x) {
        for (int i = r + 1; i <= n; i += i & -i) bit[i] += x;
    }
    // busca binaria para o maior indice i (i < n) tal que qry(i) < x
    int bs(int x) {
        int i = 0, k = 0;
        while (1 << (k + 1) <= n) k++;
        while (k >= 0) {
            int nxt_i = i + (1 << k);
            if (nxt_i <= n && bit[nxt_i] < x) {

```

```
                i = nxt_i;
                x -= bit[i];
            }
            k--;
        }
        return i - 1;
    }
};
```

7.3 MinQueue (ed1384)

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> pi;
#define ff first
#define ss second
// push, pop: O(1) arrotizado e get_min: O(1)
// Util em problemas de sliding window ou semelhantes
// A ideia e guardar apenas os elementos necessarios para encontrar o minimo
struct MinQueue {
    deque<pi> dq;
    int added = 0, removed = 0;
    // adiciona x no final da fila
    void push(int x) {
        // x > dq.back().ff para MaxQueue
        while (!dq.empty() and x < dq.back().ff) {
            dq.pop_back();
        }
        dq.push_back({x, added});
        added++;
    }
    // remove o elemento do inicio da final
    void pop() { // pop(int x) -> remove x
        // if (!dq.empty() and dq.front().ss == x) {
        if (!dq.empty() and dq.front().ss == removed) {
            dq.pop_front();
        }
        removed++;
    }
    int get_min() {
        return dq.front().ff;
    }
};
// https://noic.com.br/materiais-informatica/curso/data-structures-01/
// https://cp-algorithms.com/data_structures/stack_queue_modification.html
```

7.4 MinQueueAggStack (3748a1)

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> pi;
#define ff first
#define ss second
// Aggregated Queue e Stack
// - Esse codigo pode ser alterado para min, max, gcd, or, ...
// - E mais flexivel que a MinQueue com deque do arquivo MinQueue.cpp
// - Toda fila pode ser implementada como duas pilhas
// modifique o op para a operacao que vc quer (op deve ser agregavel)
int op(int a, int b) { return a | b; }
// int op(int a, int b) { return min(a, b); }
struct AggStack {
    // no second do pair tem o valor agregado da operacao
    stack<pi> st;
    // adiciona um novo elemento (O(1))
    void push(int x) {
        int cur = st.empty() ? x : op(st.top().ss, x);
        st.push({x, cur});
    }
    // remove o elemento do topo (O(1))
    void pop() { st.pop(); }
    // retorna o valor agregado do op (minimo, or, ...)
    int agg() { return st.top().ss; }
};
```

```

// checa se a pilha esta vazia
bool empty() { return st.empty(); }
// retorna o topo da pilha
pi top() { return st.top(); }
};
struct AggQueue {
    AggStack in, out;
    // adiciona um elemento na fila (O(1) amortizado)
    void push(int x) { in.push(x); }
    // remove o primeiro elemento (O(1) amortizado)
    void pop() {
        if (out.empty()) {
            while (!in.empty()) {
                auto [x, cur] = in.top();
                in.pop();
                out.push(x);
            }
        }
        out.pop();
    }
    // retorna o valor agregado da fila (O(1))
    int query() {
        if (in.empty()) return out.agg();
        if (out.empty()) return in.agg();
        return op(in.agg(), out.agg());
    }
};
// https://codeforces.com/blog/entry/143351

```

7.5 Mo (77aa1b)

```

#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5 + 10;
int v[MAXN];
// O((N + Q)*SQRT(N)) -> Offline Range Queries
int block_size; // block_size = teto(sqrt(n))
// TODO: Criar as EDs do problema
int freq[MAXN], cnt_freq[MAXN], freq_mode;
struct Query {
    int l, r, idx;
    bool operator<(Query o) const {
        return make_pair(l / block_size, r) <
               make_pair(o.l / block_size, o.r);
    }
};
void add(int idx) {
    int x = v[idx];
    cnt_freq[freq[x]]--;
    freq[x]++;
    cnt_freq[freq[x]]++;
    if (freq[x] > freq_mode) {
        freq_mode = freq[x];
    }
}
void remove(int idx) {
    int x = v[idx];
    if (freq[x] == freq_mode and cnt_freq[freq_mode] == 1) {
        freq_mode--;
    }
    cnt_freq[freq[x]]--;
    freq[x]--;
    cnt_freq[freq[x]]++;
}
int get_answer() {
    return freq_mode;
}
vector<int> mo(vector<Query> qrys) {
    vector<int> ans(qrys.size());
    sort(qrys.begin(), qrys.end());
    // TODO: Inicializar as EDs do problema
    int cur_l = 0, cur_r = -1;
    for (auto [l, r, idx] : qrys) {
        while (cur_l > l) {

```

```

            cur_l--;
            add(cur_l);
        }
        while (cur_r < r) {
            cur_r++;
            add(cur_r);
        }
        while (cur_l < l) {
            remove(cur_l);
            cur_l++;
        }
        while (cur_r > r) {
            remove(cur_r);
            cur_r--;
        }
        ans[idx] = get_answer();
    }
    return ans;
}
signed main() {
    int n, q; cin >> n, q;
    block_size = sqrt(n);
    vector<Query> qrys(q);
    for (int i = 0; i < q; i++) {
        int l, r; cin >> l >> r; l--, r--;
        qrys[i] = {l, r, i};
    }
    vector<int> ans = mo(qrys);
    for (auto x : ans) cout << x << endl;
}
// https://cp-algorithms.com/data_structures/sqrt_decomposition.html
// Mo para encontrar a maior frequencia em [l, r]
// Podemos usar esse codigo para problema de soma, minimo, maximo, frequencia,
// valores que obedecem uma condicao, etc...

```

7.6 ModInt (7a26a6)

```

#include <bits/stdc++.h>
using namespace std;
#define int long long
const int MOD = 1e9 + 7;
const int MAXN = 5e5 + 5;
struct modint {
    using m = modint;
    int val;
    modint(int v = 0) { val = v % MOD; }
    int pow(int y) {
        m x = val, res = 1;
        while (y) {
            if (y & 1)
                res *= x;
            x *= x;
            y >>= 1;
        }
        return res.val;
    }
    int inv() { return pow(MOD - 2); }
    void operator=(int o) { val = o % MOD; }
    void operator=(m o) { val = o.val % MOD; }
    void operator+=(m o) { *this = *this + o; }
    void operator-=(m o) { *this = *this - o; }
    void operator*=(m o) { *this = *this * o; }
    void operator/=(m o) { *this = *this / o; }
    bool operator==(m o) { return val == o.val; }
    bool operator!=(m o) { return val != o.val; }
    int operator*(m o) { return ((val * o.val) % MOD); }
    int operator/(m o) { return (val * o.inv()) % MOD; }
    int operator+(m o) { return (val + o.val) % MOD; }
    int operator-(m o) { return (val - o.val + MOD) % MOD; }
    friend istream& operator >>(istream& in, m& a) {
        int val; in >> val; a = m(val); return in;
    }
    friend ostream& operator <<(ostream& out, m a) {
        return out << a.val;
    }
}

```

```
};
modint fat[MAXN], inv[MAXN], invfat[MAXN];
void pre() {
    // Calculando Fatorial
    fat[0] = 1;
    for (int i = 1; i < MAXN; i++)
        fat[i] = fat[i-1] * i;
    // Calculando Inverso Fatorial
    inv[1] = 1;
    for (int i = 2; i < MAXN; i++) {
        int val = MOD / i;
        val = (inv[MOD % i] * val) % MOD;
        val = MOD - val;
        inv[i] = val;
    }
    invfat[0] = 1;
    invfat[MAXN - 1] = modint(fat[MAXN - 1]).inv();
    for (int i = MAXN - 2; i >= 1; i--)
        invfat[i] = invfat[i + 1] * (i + 1);
}
// C(n, k) = n! / (k! * (n - k)!)
modint comb(int n, int k) {
    modint ans = fat[n] * invfat[k];
    ans *= invfat[n - k];
    return ans;
}
// A(n, k) = n! / (n - k)!
modint arr(int n, int k) {
    modint ans = fat[n] * invfat[n - k];
    return ans;
}
}
signed main() {
    pre();
    modint a = 10, b = 3; // substitua o MOD para 7
    cout << a + b << endl; // 6
    cout << a - b << endl; // 0
    cout << a * b << endl; // 2
    cout << a / b << endl; // 1
    cout << a.pow(5) << endl; // 5
    cout << comb(5, 2) << endl; // 10
    cout << arr(5, 2) << endl; // 20
}
```

7.7 OrderStatisticSet (1234d2)

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
// Consiste em uma Arvore Binaria Balanceada (std::set + superpoderes)
// Armazena elementos unicos e de forma ordenada
// Operacoes:
// 1 - Todas de um std::set
// 2 - find_by_order(k): retorna um iterator para o k-esimo valor (0-INDEXXED) (O(log(n)))
// 3 - order_of_key(k): retorna o numero de elementos estritamente menores
// que k. (ou seja, o NUMERO DE INVERSOES)
template <class T>
using ordered_set = tree<
    T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
// Voce pode fazer um multiset com ordered_set<pair<T, int>>, o segundo elemento
// e um indice por exemplo.
signed main() {
    ordered_set<int> s;
    // Insercao
    for (int i = 0; i < 10; i++) s.insert(i);
    // Leitura
    for (auto i : s) cout << i << endl;
    int k = 0;
    // Posicao do valor k
    cout << *s.find_by_order(k) << endl;
    // Quantos sao menores do que k O(log|s|):
    cout << s.order_of_key(k) << endl;
```

```
// Remocao - Set
s.erase(5); // apaga o valor 5 (se existir)
// apaga o elemento pelo iterator
auto it = s.find(7);
if (it != s.end()) s.erase(it);
// remove todos os menores < 5
auto it_end = s.lower_bound(5);
while (s.begin() != it_end) {
    s.erase(s.begin());
}
}
// https://codeforces.com/blog/entry/123624
```

7.8 SegmentTree (b19a9b)

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
const int INF = 0x3f3f3f3f;
#define MAXN 1000000
int n, v[MAXN], seg[4*MAXN]; // a SegTree esta 0-INDEXXED
// Funcoes de Apoio
int single(int x) {return x;}
int neutral() {return -INF;}
int merge(int a, int b) {return max(a, b);}
// p -> indice na segtree, [l, r] -> intervalo da subarray
int build(int p=1, int l=0, int r=n-1) { // O(n)
    if (l == r) return seg[p] = single(v[l]);
    int m = (l+r)/2;
    return seg[p] = merge(build(2*p, l, m), build(2*p+1, m+1, r));
}
// query no intervalo [a, b]
int qry(int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or a > r) return neutral();
    if (a <= l and r <= b) return seg[p];
    int m = (l+r)/2;
    return merge(qry(a, b, 2*p, l, m), qry(a, b, 2*p+1, m+1, r));
}
// update -> v[i] = x
int upd(int i, int x, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (i < l or r < i) return seg[p];
    if (l == r) return seg[p] = single(x);
    int m = (l+r)/2;
    return seg[p] = merge(upd(i, x, 2*p, l, m), upd(i, x, 2*p+1, m+1, r));
}
// primeira posicao >= x em [a, b] (ou -1, caso nao exista)
// Obs.: So funciona na SegTree de Maximos
int first_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
    if (l == r) return l;
    int m = (l+r)/2;
    int left = first_above(x, a, b, 2*p, l, m);
    if (left != -1) return left;
    return first_above(x, a, b, 2*p+1, m+1, r);
}
// ultima posicao >= x em [a, b] (ou -1, caso nao exista)
// Obs.: So funciona na SegTree de Maximos
int last_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
    if (l == r) return l;
    int m = (l+r)/2;
    int right = last_above(x, a, b, 2*p+1, m+1, r);
    if (right != -1) return right;
    return last_above(x, a, b, 2*p, l, m);
}
// retorna a posicao i do vetor do Kth l
// Obs.: So funciona na SegTree de Soma
// k e 1-INDEXXED e o RETORNO e 0-INDEXXED
int find_kth(int k, int p=1, int l=0, int r=n-1) {
    if (k > seg[p]) return -1;
    if (l == r) return l;
    int m = (l + r) / 2;
    if (seg[p*2] >= k)
        return find_kth(k, p*2, l, m);
    else
```

```

        return find_kth(k - seg[p*2], p*2+1, m+1, r);
    }
    signed main() {
        ios_base::sync_with_stdio(0); cin.tie(0);
        cin >> n;
        // Voce pode inicializar v[] com um valor neutro
        // fill(v, v+MAXN, neutral());
        // Leitura de v[]
        for(int i = 0; i < n; i++) cin >> v[i];
        // Construir a SegTree
        build(1, 0, n - 1);
        // Query do intervalo a, b
        int a, b; cin >> a >> b;
        cout << qry(a, b, 1, 0, n - 1) << endl;
        // Update de v[i] = x
        int i, x; cin >> i >> x;
        cout << upd(i, x, 1, 0, n - 1) << endl;
    }
    /* Segment Tree ou SegTree ou Arvore de Segmentos
    # Altura da arvore -> O(log(n))
    # Qtd de nos -> 2n - 1 -> O(n)
    -> Problemas de RMQ: Range Minimum Query
        -> Descobrir o minimo de um intervalo [l, r]

    -> Soma, Minimo, Maximo, Produto, ...
    -> SegTree para qualquer operacao ASSOCIATIVA: (A ? B) ? C = A ? (B ? C)
    */

```

7.9 SegmentTreeLazy (e2aa98)

```

#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define MAXN 100005
int n, q, v[MAXN], seg[MAXN*4], lazy[MAXN*4], leafs[MAXN];
// Funcoes de Apoio
int single(int x) { return x; }
int neutral() { return 0; }
int merge(int a, int b) { return a + b; }
// Funcao de Propagacao
void prop(int p, int l, int r) { // O(1)
    seg[p] += lazy[p]*(r-l+1);
    if (l != r) {
        lazy[2*p] += lazy[p];
        lazy[2*p+1] += lazy[p];
    }
    lazy[p] = 0;
}
// p -> indice na segtree, [l, r] -> intervalo da subarray
int build(int p=1, int l=0, int r=n-1) { // O(n)
    lazy[p] = 0;
    if (l == r) return seg[p] = single(v[l]);
    int m = (l+r)/2;
    return seg[p] = merge(build(2*p, l, m), build(2*p+1, m+1, r));
}
// query no intervalo [a, b]
int qry(int a, int b, int p=1, int l=0, int r=n-1) {
    prop(p, l, r);
    if (b < l or a > r) return neutral();
    if (a <= l and r <= b) return seg[p];
    int m = (l+r)/2;
    return merge(qry(a, b, 2*p, l, m), qry(a, b, 2*p+1, m+1, r));
}
// update -> Para todo v[i] += x, com a <= i <= b.
int upd(int a, int b, int x, int p=1, int l=0, int r=n-1) {
    prop(p, l, r);
    if (a <= l and r <= b) {
        lazy[p] += x;
        prop(p, l, r);
        return seg[p];
    }
    if (b < l or r < a) return seg[p];
    int m = (l+r)/2;
    return seg[p] = merge(upd(a, b, x, 2*p, l, m), upd(a, b, x, 2*p+1, m+1, r));
}

```

```

}
// Funcao para visitar as folhas, ou seja, o v[]
void get_leafs(int p = 1, int l = 0, int r = n - 1) { // O(n)
    prop(p, l, r);
    if (l == r) {
        leafs[l] = seg[p];
        return;
    }
    int m = (l+r)/2;
    get_leafs(2*p, l, m);
    get_leafs(2*p+1, m+1, r);
}
// primeira posicao >= x em [a, b] (ou -1, caso nao exista)
// Obs.: So funciona na SegTree de Maximos
int first_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
    if (l == r) return l;
    int m = (l+r)/2;
    int left = first_above(x, a, b, 2*p, l, m);
    if (left != -1) return left;
    return first_above(x, a, b, 2*p+1, m+1, r);
}
// ultima posicao >= x em [a, b] (ou -1, caso nao exista)
// Obs.: So funciona na SegTree de Maximos
int last_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
    if (l == r) return l;
    int m = (l+r)/2;
    int right = last_above(x, a, b, 2*p+1, m+1, r);
    if (right != -1) return right;
    return last_above(x, a, b, 2*p, l, m);
}
}

```

8 Techniques

8.1 CoordinateCompression (eca1bb)

```

#include <bits/stdc++.h>
using namespace std;
// Considere que 1 <= ai <= 10^9 e 1 <= |a| <= 10^5.
// Vamos comprimir os valores de ai para [0, 10^5-1]. -> 0-INDEXED
map<int, int> coord; // mantem a propriedade de ordenacao dos valores
void coordinate_compression(vector<int> &v) {
    set<int> s; for (auto x : v) s.insert(x);
    int idx = 0;
    for (auto x : s) {
        coord[x] = idx;
        idx++;
    }
    for (int i = 0; i < (int) v.size(); i++) {
        v[i] = coord[v[i]];
    }
}

```

8.2 Grid (ed0740)

```

#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define ff first
#define ss second
typedef pair<int, int> pi;
const int MAX = 1e3;
int n, m;
char grid[MAX][MAX];
bool vis[MAX][MAX];
// Movimentos: cima, baixo, esquerda, direita
pi mov[16] = {
    // Esquerda, Direita, Cima, Baixo

```



```

    {-1, 0}, {1, 0}, {0, -1}, {0, 1},
    // Diagonais
    {-1, -1}, {1, 1}, {-1, 1}, {1, -1},
    // Movimento de cavalo
    {-2, -1}, {2, 1}, {-2, 1}, {2, -1},
    {-1, -2}, {1, 2}, {-1, 2}, {1, -2}
};
// Funcao para validar de new_i, new_j sao validos
bool valid(int i, int j) {
    // Podemos adicionar outras CONDICoes...
    // Por exemplo, "Ja foi visitado", "movimento nao e obstruido"
    return i >= 0 and j >= 0 and i < n and j < m and !vis[i][j];
}
signed main() {
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> grid[i][j];
        }
    }
    int i = 0, j = 0; // Posicao Inicial
    for (auto [di, dj] : mov) {
        int ni = i + di, nj = j + dj;
        if (valid(ni, nj)) {
            vis[ni][nj] = true;
            // Faca algo!
        }
    }
}

```

9 Teoria

9.1 STL

Os templates da STL são estruturas de dados e algoritmos já implementados em C++ que facilitam as implementações, além de serem muito eficientes. Em geral, todas estão incluídas no cabeçalho `<bits/stdc++.h>`. As estruturas são templates genéricos, podendo ser usadas com qualquer tipo; todos os exemplos a seguir são com `int` apenas por motivos de simplicidade.

9.1.1 Vector

Um vetor dinâmico (que pode crescer e diminuir de tamanho).

- `vector<int> v(n, x)`: Cria um vetor de inteiros com n elementos inicializados com $x - O(n)$.
- `v.push_back(x)`: Adiciona o elemento x no final do vetor $- O(1)$.
- `v.pop_back()`: Remove o último elemento do vetor $- O(1)$.
- `v.size()`: Retorna o tamanho do vetor $- O(1)$.
- `v.empty()`: Retorna `true` se o vetor estiver vazio $- O(1)$.
- `v.clear()`: Remove todos os elementos do vetor $- O(n)$.
- `v.front()`: Retorna o primeiro elemento do vetor $- O(1)$.
- `v.back()`: Retorna o último elemento do vetor $- O(1)$.
- `v.begin()`: Retorna um iterador para o primeiro elemento do vetor $- O(1)$.

- `v.end()`: Retorna um iterador para o elemento seguinte ao último do vetor $- O(1)$.
- `v.insert(it, x)`: Insere o elemento x na posição apontada pelo iterador $it - O(n)$.
- `v.erase(it)`: Remove o elemento apontado pelo iterador $it - O(n)$.
- `v.erase(it1, it2)`: Remove elementos no intervalo $[it1, it2) - O(n)$.
- `v.resize(n)`: Redimensiona o vetor para n elementos $- O(n)$.
- `v.resize(n, x)`: Redimensiona o vetor para n elementos, todos inicializados com $x - O(n)$.

9.1.2 Pair

Um par de elementos (de tipos possivelmente diferentes).

- `pair<int, int> p`: Cria um par de inteiros $- O(1)$.
- `p.first`: Retorna o primeiro elemento do par $- O(1)$.
- `p.second`: Retorna o segundo elemento do par $- O(1)$.

9.1.3 Set

Um conjunto de elementos únicos. Por baixo, é uma árvore de busca binária balanceada.

- `set<int> s`: Cria um conjunto de inteiros $- O(1)$.
- `s.insert(x)`: Insere o elemento x no conjunto $- O(\log n)$.
- `s.erase(x)`: Remove o elemento x do conjunto $- O(\log n)$.
- `s.find(x)`: Retorna um iterador para x ; se x não existir, retorna `s.end()` $- O(\log n)$.
- `s.size()`: Retorna o tamanho do conjunto $- O(1)$.
- `s.empty()`: Retorna `true` se estiver vazio $- O(1)$.
- `s.clear()`: Remove todos os elementos $- O(n)$.
- `s.begin()`: Retorna um iterador para o primeiro elemento $- O(1)$.
- `s.end()`: Retorna um iterador para após o último elemento $- O(1)$.

9.1.4 Multiset

Basicamente um `set`, mas permite elementos repetidos. Possui todos os métodos de um `set`.

Declaração: `multiset<int> ms`.

Um detalhe é que, ao usar o método `erase`, ele remove todas as ocorrências do elemento. Para remover apenas uma ocorrência, usar `ms.erase(ms.find(x))`.

9.1.5 Map

Um conjunto de pares chave-valor, onde as chaves são únicas. Por baixo, é uma árvore de busca binária balanceada.

- `map<int, int> m`: Cria um mapa de inteiros para inteiros – $O(1)$.
- `m[key]`: Retorna o valor associado à chave `key` – $O(\log n)$.
- `m[key] = value`: Associa o valor `value` à chave `key` – $O(\log n)$.
- `m.erase(key)`: Remove a chave do mapa – $O(\log n)$.
- `m.find(key)`: Retorna um iterador para o par chave-valor com chave `key`, ou `m.end()` se não existir – $O(\log n)$.
- `m.size()`: Retorna o tamanho do mapa – $O(1)$.
- `m.empty()`: Retorna `true` se estiver vazio – $O(1)$.
- `m.clear()`: Remove todos os pares chave-valor – $O(n)$.
- `m.begin()`: Retorna um iterador para o primeiro par chave-valor – $O(1)$.
- `m.end()`: Retorna um iterador para após o último par chave-valor – $O(1)$.

9.1.6 Queue

Uma fila (primeiro a entrar, primeiro a sair).

- `queue<int> q`: Cria uma fila de inteiros – $O(1)$.
- `q.push(x)`: Adiciona o elemento `x` no final da fila – $O(1)$.
- `q.pop()`: Remove o primeiro elemento da fila – $O(1)$.
- `q.front()`: Retorna o primeiro elemento da fila – $O(1)$.
- `q.size()`: Retorna o tamanho da fila – $O(1)$.
- `q.empty()`: Retorna `true` se a fila estiver vazia – $O(1)$.

9.1.7 Priority Queue

Uma fila de prioridade (o maior elemento é o primeiro a sair).

- `priority_queue<int> pq`: Cria uma fila de prioridade de inteiros – $O(1)$.
- `pq.push(x)`: Adiciona o elemento `x` na fila de prioridade – $O(\log n)$.
- `pq.pop()`: Remove o maior elemento da fila – $O(\log n)$.
- `pq.top()`: Retorna o maior elemento da fila – $O(1)$.
- `pq.size()`: Retorna o tamanho da fila de prioridade – $O(1)$.
- `pq.empty()`: Retorna `true` se a fila estiver vazia – $O(1)$.

Para fazer uma fila de prioridade em que o menor elemento sai primeiro, use `priority_queue<int, vector<int>, greater<int>> pq`.

9.1.8 Stack

Uma pilha (último a entrar, primeiro a sair).

- `stack<int> s`: Cria uma pilha de inteiros – $O(1)$.
- `s.push(x)`: Adiciona o elemento `x` no topo da pilha – $O(1)$.
- `s.pop()`: Remove o elemento do topo da pilha – $O(1)$.
- `s.top()`: Retorna o elemento do topo da pilha – $O(1)$.
- `s.size()`: Retorna o tamanho da pilha – $O(1)$.
- `s.empty()`: Retorna `true` se a pilha estiver vazia – $O(1)$.

9.1.9 Bitset

Um conjunto de bits, serve para representar máscaras quando um inteiro não é suficiente. Possui operações bitwise otimizadas pelo processador.

- `bitset<N> a`: Cria um bitset de tamanho `N` (`N` deve ser constante) – $O(1)$.
- `a[i]`: Retorna o valor do bit na posição `i` – $O(1)$.
- `a.count()`: Retorna o número de bits 1 no bitset – $O(N/w)$.
- `a.size()`: Retorna o tamanho do bitset – $O(1)$.
- `a.set(i)`: Seta o bit na posição `i` para 1 – $O(1)$.
- `a.set()`: Seta todos os bits para 1 – $O(N/w)$.
- `a.reset(i)`: Seta o bit na posição `i` para 0 – $O(1)$.
- `a.reset()`: Seta todos os bits para 0 – $O(N/w)$.
- `a.flip(i)`: Inverte o bit na posição `i` – $O(1)$.
- `a.flip()`: Inverte todos os bits – $O(N/w)$.
- `a.to_ulong()`: Retorna o valor do bitset como um inteiro – $O(N/w)$.

O bitset também suporta operações como `&`, `|`, `^`, `~`, `<<`, `>>`, entre outros.

Obs: `w` é o tamanho da palavra do processador, em geral 32 ou 64 bits.

9.1.10 Funções úteis

- `min(a, b)`: Retorna o menor entre `a` e `b` – $O(1)$.
- `max(a, b)`: Retorna o maior entre `a` e `b` – $O(1)$.
- `abs(a)`: Retorna o valor absoluto de `a` – $O(1)$.
- `swap(a, b)`: Troca os valores de `a` e `b` – $O(1)$.
- `sqrt(a)`: Retorna a raiz quadrada de `a` – $O(\log a)$.
- `ceil(a)`: Retorna o menor inteiro maior ou igual a `a` – $O(1)$.
- `floor(a)`: Retorna o maior inteiro menor ou igual a `a` – $O(1)$.
- `round(a)`: Retorna o inteiro mais próximo de `a` – $O(1)$.

9.1.11 Funções úteis para vetores

Para usar em `std::vector`, sempre passar `v.begin()` e `v.end()` como argumentos para essas funções.

Se for um vetor estilo C, usar `v` e `v + n`. Exemplo:

```
int v[10];    sort(v, v + 10);
```

Lembrete: `v.end()` é um iterador para o elemento seguinte ao último do vetor, então não é um iterador válido.

As funções de vetor em geral são da forma `[L, R)`, ou seja, `L` é incluso e `R` é exclusivo.

- `fill(v.begin(), v.end(), x)`: Preenche o vetor `v` com o valor `x` – $O(n)$.
- `sort(v.begin(), v.end())`: Ordena o vetor `v` – $O(n \log n)$.
- `reverse(v.begin(), v.end())`: Inverte o vetor `v` – $O(n)$.
- `accumulate(v.begin(), v.end(), 0)`: Soma todos os elementos do vetor `v` – $O(n)$.
- `max_element(v.begin(), v.end())`: Retorna um iterador para o maior elemento do vetor `v` – $O(n)$.
- `min_element(v.begin(), v.end())`: Retorna um iterador para o menor elemento do vetor `v` – $O(n)$.
- `count(v.begin(), v.end(), x)`: Retorna o número de ocorrências do elemento `x` no vetor `v` – $O(n)$.
- `find(v.begin(), v.end(), x)`: Retorna um iterador para a primeira ocorrência de `x` no vetor `v`, ou `v.end()` se não existir – $O(n)$.
- `lower_bound(v.begin(), v.end(), x)`: Retorna um iterador para o primeiro elemento maior ou igual a `x`; o vetor deve estar ordenado – $O(\log n)$.
- `upper_bound(v.begin(), v.end(), x)`: Retorna um iterador para o primeiro elemento estritamente maior que `x`; o vetor deve estar ordenado – $O(\log n)$.
- `next_permutation(a.begin(), a.end())`: Rearranja os elementos do vetor `a` para a próxima permutação lexicograficamente maior – $O(n)$.

9.1.12 Pragmas

Os pragmas são diretivas para o compilador, que podem ser usadas para otimizar o código.

Temos os pragmas de otimização, como por exemplo:

- `#pragma GCC optimize("O2")`: Otimizações de nível 2 (padrão de competições)

- `#pragma GCC optimize("O3")`: Otimizações de nível 3 (seguro para usar)
- `#pragma GCC optimize("Ofast")`: Otimizações agressivas (perigoso!)
- `#pragma GCC optimize("unroll-loops")`: Otimiza os loops mas pode levar a *cache misses*

E também os pragmas de *target*, que são usados para otimizar o código para um certo processador:

- `#pragma GCC target("avx2")`: Otimiza instruções para processadores com suporte a AVX2
- `#pragma GCC target("sse4")`: Parecido com o de cima, mas mais antigo
- `#pragma GCC target("popcnt")`: Otimiza o *popcount* em processadores que suportam
- `#pragma GCC target("lzcnt")`: Otimiza o *leading zero count* em processadores que suportam
- `#pragma GCC target("bmi")`: Otimiza instruções de *bit manipulation* em processadores que suportam
- `#pragma GCC target("bmi2")`: Mesmo que o de cima, mas mais recente

Em geral, esses pragmas são usados para otimizar o código em competições, mas é importante usá-los com certa sabedoria, em alguns casos eles podem piorar o desempenho do código.

Uma opção relativamente segura de se usar é a seguinte:

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

9.1.13 Constantes em C++

Constante	Nome em C++	Valor
π	<code>M_PI</code>	3.141592...
$\pi/2$	<code>M_PI_2</code>	1.570796...
$\pi/4$	<code>M_PI_4</code>	0.785398...
$1/\pi$	<code>M_1_PI</code>	0.318309...
$2/\pi$	<code>M_2_PI</code>	0.636619...
$2/\sqrt{\pi}$	<code>M_2_SQRTPI</code>	1.128379...
$\sqrt{2}$	<code>M_SQRT2</code>	1.414213...
$1/\sqrt{2}$	<code>M_SQRT1_2</code>	0.707106...
e	<code>M_E</code>	2.718281...
$\log_2 e$	<code>M_LOG2E</code>	1.442695...
$\log_{10} e$	<code>M_LOG10E</code>	0.434294...
$\ln 2$	<code>M_LN2</code>	0.693147...
$\ln 10$	<code>M_LN10</code>	2.302585...

9.2 Matemática

9.2.1 Definições

Algumas definições e termos importantes:

Funções

- **Comutativa:** Uma função $f(x, y)$ é comutativa se $f(x, y) = f(y, x)$.
- **Associativa:** Uma função $f(x, y)$ é associativa se $f(x, f(y, z)) = f(f(x, y), z)$.
- **Idempotente:** Uma função $f(x, y)$ é idempotente se $f(x, x) = x$.

9.2.2 Números primos

Números primos são muito úteis para funções de hashing (dentre outras coisas). Aqui vão vários números primos interessantes:

Primos com truncamento à esquerda Números primos tais que qualquer sufixo deles é um número primo:

33,333,31
357,686,312,646,216,567,629,137

Primos gêmeos (Twin Primes) Pares de primos da forma $(p, p + 2)$ (aqui tem só alguns pares aleatórios, existem muitos outros).

Números primos de Mersenne São os números primos da forma $2^m - 1$, onde m é um número inteiro positivo.

9.2.3 Operadores lineares

Rotação no sentido anti-horário por θ

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Reflexão em relação à reta $y = mx$

$$\frac{1}{m^2 + 1} \begin{bmatrix} 1 - m^2 & 2m \\ 2m & m^2 - 1 \end{bmatrix}$$

Inversa de uma matriz 2×2 A

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Cisalhamento horizontal por K

$$\begin{bmatrix} 1 & K \\ 0 & 1 \end{bmatrix}$$

Cisalhamento vertical por K

$$\begin{bmatrix} 1 & 0 \\ K & 1 \end{bmatrix}$$

Mudança de base $\vec{a}_\beta$ são as coordenadas do vetor $\vec{a}$ na base β . $\vec{a}$ são as coordenadas do vetor $\vec{a}$ na base canônica.

$\vec{b}_1$ e $\vec{b}_2$ são os vetores de base para β .

C é uma matriz que muda da base β para a base canônica.

$$C\vec{a}_\beta = \vec{a}$$

$$C^{-1}\vec{a} = \vec{a}_\beta$$

$$C = \begin{bmatrix} b_{1x} & b_{2x} \\ b_{1y} & b_{2y} \end{bmatrix}$$

Propriedades das operações de matriz

$$(AB)^{-1} = B^{-1}A^{-1}$$

$$(AB)^T = B^T A^T$$

$$(A^{-1})^T = (A^T)^{-1}$$

$$(A + B)^T = A^T + B^T$$

$$\det(A) = \det(A^T)$$

$$\det(AB) = \det(A) \det(B)$$

Seja A uma matriz $N \times N$:

$$\det(kA) = k^N \det(A)$$

9.2.4 Sequências numéricas

Sequência de Fibonacci Primeiros termos: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Definição:

$$F_0 = F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

Matriz de recorrência:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} F_{n-2} \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} F_{n-1} \\ F_n \end{bmatrix}$$

Sequência de Catalan Primeiros termos: 1, 1, 2, 5, 14, 42, 132, 429, 1430, ...

Definição:

$$C_0 = C_1 = 1$$

$$C_n = \sum_{i=0}^{n-1} C_i \cdot C_{n-1-i}$$

Definição analítica:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Propriedades úteis:

- C_n é o número de árvores binárias com $n+1$ folhas.
- C_n é o número de sequências de parênteses bem formadas com n pares de parênteses.

9.2.5 Análise combinatória

Fatorial O fatorial de um número n é o produto de todos os inteiros positivos menores ou iguais a n .

O fatorial conta o número de permutações de n elementos.

$$n! = n \cdot (n-1)!$$

Em particular, $0! = 1$.

Combinação Conta o número de maneiras de escolher k elementos de um conjunto de n elementos.

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Arranjo Conta o número de maneiras de escolher k elementos de um conjunto de n elementos, onde a ordem importa.

$$P(n, k) = \frac{n!}{(n-k)!}$$

Estrelas e barras Conta o número de maneiras de distribuir n elementos idênticos em k recipientes distintos.

$$\binom{n+k-1}{k-1}$$

Princípio da inclusão-exclusão O princípio da inclusão-exclusão é uma técnica para contar o número de elementos em uma união de conjuntos.

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \dots < i_k \leq n} |A_{i_1} \cap \dots \cap A_{i_k}|$$

Princípio da casa dos pombos Se n pombos são colocados em m casas, então pelo menos uma casa terá $\lceil \frac{n}{m} \rceil$ pombos ou mais.

9.2.6 Teoria dos números

Pequeno teorema de Fermat Se p é um número primo e a é um inteiro não divisível por p , então $a^{p-1} \equiv 1 \pmod{p}$.

Teorema de Euler Se m e a são inteiros positivos coprimos, então $a^{\phi(m)} \equiv 1 \pmod{m}$, onde $\phi(m)$ é a função totiente de Euler.

Aritmética modular Quando estamos trabalhando com aritmética módulo um número p , todos os valores existentes estão entre $[0, p-1]$.

Algumas propriedades e equivalências úteis para usar aritmética modular em código são:

- $(a+b) \bmod p \equiv ((a \bmod p) + (b \bmod p)) \bmod p$
- $(a-b) \bmod p \equiv ((a \bmod p) - (b \bmod p)) \bmod p$
 - Note que o resultado pode ser negativo, nesse caso é necessário adicionar p ao resultado. De forma geral, geralmente fazemos $(a-b+p) \bmod p$ (assumindo que a e b já estão no intervalo $[0, p-1]$).
- $(a \cdot b) \bmod p \equiv ((a \bmod p) \cdot (b \bmod p)) \bmod p$
- $a^b \bmod p \equiv ((a \bmod p)^b) \bmod p$
- $a^b \bmod p \equiv ((a \bmod p)^{(b \bmod \phi(p))}) \bmod p$ se a e p são coprimos

9.3 Grafos

9.3.1 Definições Iniciais

- **Grafo:** Um grafo é um conjunto de vértices e um conjunto de arestas que conectam os vértices. Formalmente, $G(V, E)$ onde V é o conjunto de vértices e E o conjunto de arestas.
- **Caminho:** Um caminho P de x_0 para x_k é um subgrafo não vazio da forma $V = \{x_0, x_1, \dots, x_k\}$ e $E = \{\{x_0, x_1\}, \{x_1, x_2\}, \dots, \{x_{k-1}, x_k\}\}$ em que todos os x_i são distintos.
- **Comprimento de um caminho:** Definido como o número de arestas em E .
- **Vizinho:** Para cada vértice, os vértices que podemos atingir a partir das suas arestas são conhecidos como vizinhos.
- **Grau:** O número de vizinhos de um vértice.

9.3.2 Ciclos

- **Ciclo:** Sendo P um caminho de x_0 para x_k de comprimento ≥ 2 , um ciclo é definido como $P + \{x_k, x_0\}$, ou seja, adiciona uma aresta fechando o ciclo.
- **Ciclo Simples:** É um tipo especial de ciclo onde:
 - Nenhum vértice (além do inicial e final) se repete.
 - Nenhuma aresta se repete.
 - Tem ao menos 3 vértices distintos (em grafos simples), ou seja, comprimento ≥ 3 .

9.3.3 Conectividade

- **Grafo Conexo:** Um grafo é conexo se existe um caminho entre todos os pares de vértices. Caso contrário, o grafo é desconexo.
- **Componente Conexo:** Um subgrafo conexo maximal de um grafo G .
- **Grafo Bipartido:** Um grafo é bipartido se é possível dividir os vértices em dois conjuntos disjuntos de forma que todas as arestas conectem um vértice de um conjunto com um vértice do outro conjunto, ou seja, não existem arestas que conectem vértices do mesmo conjunto.

9.3.4 Árvores

- **Árvore:** Um grafo acíclico (sem ciclo) e conexo.
- **Folhas:** Vértices de grau 1.
- **Raiz:** Uma árvore pode ser enraizada por um vértice qualquer (introduz noção de pai e filhos).
- **Árvore Geradora Mínima (AGM):** Uma árvore que conecta todos os vértices de um grafo e possui o menor custo possível, também conhecido como *Minimum Spanning Tree (MST)*.

Afirmações equivalentes para uma árvore G :

- G é uma árvore $\iff G$ é um grafo conexo e $|E| = |V| - 1$.
- G é uma árvore $\iff$ Para todo $v, w \in G$ existe EXATAMENTE UM caminho de v para w .
- G é uma árvore $\iff G$ é conexo e a remoção de qualquer aresta o torna desconexo.
- G é uma árvore $\iff G$ não contém ciclos, mas adicionar qualquer aresta a G cria um ciclo.

Diâmetro Consiste no maior caminho de uma árvore.

Algoritmo:

1. Roda BFS a partir de um vértice qualquer u para encontrar o vértice v mais distante.
2. Roda BFS a partir do vértice v para encontrar o vértice w mais distante. O caminho de v a w é o diâmetro.

10 Utils

10.1 Makefile (a879a5)

```
CXX = g++
CXXFLAGS = -fsanitize=address,undefined -fno-omit-frame-pointer -g -Wall -
Wshadow -std=c++20 -Wno-unused-result -Wno-sign-compare -Wno-char-
subscripts
```

10.2 custom hash (13b6af)

```
#include <bits/stdc++.h>
using namespace std;
// importas para o hash_table
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef long long ll;
// o unordered_map é facil de hackear:
// as operacoes ficam O(N), podemos melhorar
// isso com uma funcao de hash melhor
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    // posso alterar o int para ll e etc...
    size_t operator()(int x) const {
        static const int FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
    size_t operator()(pair<int, int> x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x.first + FIXED_RANDOM)
            ^ splitmix64(x.second + FIXED_RANDOM);
    }
};
// criando algumas estruturas "seguras"
template <typename K, typename V>
using umap = unordered_map<K, V, custom_hash>;
// o hash table é mais performático em media
template <typename K, typename V>
using ht = gp_hash_table<K, V, custom_hash>;
// https://codeforces.com/blog/entry/62393
```

10.3 hash (d78ff6)

```
# Para usar (hash das linhas [11, 12]):
# bash hash.sh arquivo.cpp 11 12
sed -n $2', '$3' p' $1 | sed '/^#w/d' | cpp -dD -P -fpreprocessed | tr -d '[:
space:]' | md5sum | cut -c-6
```

10.4 mistakes (06c097)

(dicas da lib do collares - traduzido)

Dicas Gerais de Estrategia

- * Todos devem ter lido todos os problemas ate o final da segunda hora de competicao.
- * ****Na ultima hora, apenas uma questao deve ser tentada por vez.****
- * Sempre use ***vectors*** em vez de ***arrays*** no caso unidimensional.
- * Use ***vectors*** em vez de ***arrays*** no caso bidimensional se a dimensao mais externa (numero de linhas) nao for muito grande. Isso permite uma melhor depuracao (***debugging***).

Ao Pensar (Antes de Programar)

1. ****Seja organizado(a)!****
2. Nao toque no computador a menos que a solucao esteja pronta, incluindo os detalhes de implementacao: evite pensar demais em frente ao computador.
- **O tempo gasto no papel detalhando uma solucao e tempo bem gasto.****
3. Um corolario do ponto acima:
- **Nao toque no computador se tiver duvidas sobre a sua ideia.****

Caso Voce Nao Tenha Ideias de Solucao

1. (Inclua os truques de Polya aqui) -> TODO: Gabrielzinho faca essa parte

Em Caso de ***Wrong Answer*** (Resposta Errada)

1. Certifique-se de que o algoritmo esta correto (ou seja, ****esboce uma prova de corretude****) o mais rapido possivel. Leve o tempo que precisar e verifique com cuidado.
- **Se voce tem certeza de que a ideia esta correta, nao duvide dela**** ao procurar por ***bugs***, mesmo que nao encontre nenhum erro de implementacao em lugar nenhum.
2. Se o codigo usa ***vectors***, adicione **'#define _GLIBCXX_DEBUG'** bem no ****inicio do codigo-fonte**** e submeta novamente. Um erro de execucao (***runtime error***) geralmente significa acesso fora dos limites do ***array*** (***out-of-bounds***) ou outro uso incorreto da STL.
3. Depure no papel e nao volte para o computador a cada ***bug*** que encontrar: ****verifique a solucao inteira pelo menos mais uma vez apos encontrar cada novo *bug*****.
4. Pense em casos de teste capciosos (***tricky test cases***).
5. Leia o problema novamente. Para cada restricao encontrada, verifique o codigo-fonte impresso.
6. Se voce nao conseguir encontrar nenhum ***bug*** em cinco minutos, va ao banheiro.
 - 6.1. Se ainda assim nao conseguir encontrar o ***bug***, va para outro problema e volte para a solucao errada mais tarde.
 - 6.2. Depurar um unico programa por muito tempo leva a encontrar muitos falsos ***bugs*** e faz com que seja facil deixar passar erros simples.
7. Use o **'assert(condicao)'** para verificar:

- Se um ****algoritmo construtivo**** seu bate com as condicoes que o problema pede.
- Para testar possiveis cenarios de casos de testes. Gaste um WA/RE para verificar se os casos de teste sao do jeito que voce imagina.

Em Caso de ***Runtime Error*** (Erro de Execucao)

1. Verifique possivel overflow nas operacoes.
2. Verifique divisoes e operacoes de modulo.
3. Verifique os indices dos ***arrays*** (tanto nas declaracoes quanto nos acessos).
4. Verifique se ha recursos infinitas ou ***loops*** **'while'** infinitos.

10.5 random (5b3cc9)

```
#include <bits/stdc++.h>
using namespace std;
#define all(x) (x).begin(), (x).end()
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
signed main() {
    vector<int> a(5);
    iota(all(a), 0);
    // ordena aleatoriamente
    shuffle(all(a), rng);
    // gera valor aleatorio de [0, x)
    cout << rng() % 10 << endl; // [0, 10)
    // valor aleatorio entre [a, b]
    int r = uniform_int_distribution<int>(3, 99)(rng);
    cout << r << endl;
}
```

10.6 template (c1a851)

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' //<< flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define pdbg(x) cerr << #x << " = " << x.ff << ", " << x.ss << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef long double ld;
typedef pair<int, int> pi;
// const ld EPS = 1e-9;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3f;
// const int MAXN = 2e5+5;
signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    cout.precision(10); cout.setf(ios::fixed);
}
```

10.7 vimrc (54a8c1)

```
set ts=4 sw=4 sts=4 expandtab mouse=a nu ai si undofile
```